QuickGelu Alpha Scale 多平台向量实现对比与RVV扩展指令建议

概述

分析目标: `QuickGeluAlphaScale` — QuickGelu激活函数的alpha预缩放步骤 **完整操作**: QuickGelu(x) = x * sigmoid(alpha * x) , 其中alpha通常为1.702f **基准实现**: RVV VLEN=512, VL=16 (float32), LMUL=1 **分析平台**: x86 AVX/AVX2, ARM NEON, LoongArch LSX, Power VSX, S390X, WASM SIMD **BBV数据**: 基于QEMU-BBV profiling on 独立测试可执行文件 (output/bbv_rvv512/quick-gelu/), 收益估算结合BBV热点数据与MLAS profiling热点占比约9.63%

指令方案汇总

优先级	扩展指令	来源平台	BB内收益	实现难度	RVV现状
P0	vfmul_lane	ARM NEON	减少广播开销（微架构层面）	低	RVV需vfmul_vf（每次重新广播）
P1	vfma_lane	ARM NEON	减少广播开销（微架构层面）	低	RVV需vfmacc_vf

收益计算方式（基于BBV热点数据估算）： - BB内收益 = (原BB指令数 - 扩展后BB指令数) / 原BB指令数 × 100% - 整体收益 = BB指令减少比例 × 函数执行占比（9.63%，来自perf profiling）

BBV热点数据（独立测试，VLEN=512）： - 核心循环BB: 7条指令/迭代，10,136次执行（RVV share 0.20%） - 指令序列: vsetvli → vle32 → vfmul.vf → vse32 (+地址计算) - 标量路径: fmul.s循环占6.69%，fdiv/fadd循环占5.51%（标量路径显著高于RVV路径） - 整体收益估算: BB指令减少比例 × 函数执行占比（9.63%，来自perf profiling）

基准RVV实现分析

QuickGelu完整操作结构

```
// QuickGelu(x) = x * sigmoid(alpha * x)
// 步骤1: alpha预缩放 (本RVV实现)
output[i] = input[i] * alpha;

// 步骤2: sigmoid (MlasComputeLogistic, 已向量化)
MlasComputeLogistic(output, output, count);  // sigmoid(alpha * x)

// 步骤3: 元素乘法 (MlasEltwiseMul, 已向量化)
MlasEltwiseMul(input, output, output, count);  // x * sigmoid
```

RVV Alpha Scale实现

```
// rvv_quick_gelu.inl
inline void QuickGeluAlphaScale_rvv(
    const float* input,
    float* output,
    size_t count,
    float alpha)
{
```

```
    size_t vl;
    for (size_t i = 0; i < count; ) {
        vl = __riscv_vsetvl_e32m1(count - i);
        vfloat32m1_t v = __riscv_vle32_v_f32m1(input + i, vl);  // 加载
        v = __riscv_vfmul_vf_f32m1(v, alpha, vl);               // 标量广播乘
        __riscv_vse32_v_f32m1(output + i, v, vl);               // 存储
        i += vl;
    }
}
```

RVV指令计数

操作	指令数	说明
向量加载	1	`vle32`
标量广播乘	1	`vfmul_vf` （alpha从标量寄存器广播）
向量存储	1	`vse32`
每VL元素	3	加载+乘+存储

关键观察： - RVV的`vfmul_vf`每次都从标量寄存器读取alpha并广播到所有元素 - 无lane-indexed版本（一次广播后重复使用） - MLAS CPU实现使用标量循环（未向量化），RVV已优于scalar

---

各平台对比分析

1. x86 AVX/AVX2

核心特点： - YMM/XMM寄存器，256/128位向量宽度 - `_mm256_mul_ps`：向量×向量乘法 - `_mm256_set1_ps`：标量广播（单独指令）

指令序列:

```
vmovups   ymm0, [input]       ; 加载向量
vmulps    ymm1, ymm0, ymm2    ; ymm2预先广播alpha
vmovups   [output], ymm1      ; 存储
```

操作	AVX指令数	RVV指令数
标量广播alpha	1（预设置）	0（内置于vfmul_vf）
向量加载	1	1
向量乘	1	1
向量存储	1	1
首次迭代	4	3

关键差异： - AVX需预先设置广播寄存器（`set1_ps`） - RVV的`vfmul_vf`内置广播，更简洁 - 多次迭代后AVX可复用广播寄存器，RVV每次重新广播

建议扩展：无需要。RVV在此场景已优于AVX（少1条预设置指令）。

---

2. ARM NEON

核心特点： - 128-bit Q寄存器，4×float32 - `vmulq_n_f32`：向量×标量乘法（内置广播） - **lane-indexed乘法**：`vmulq_lane_f32`，从向量中提取特定lane广播

高价值指令：

指令	功能	RVV现状
`vmulq_n_f32`	向量×标量乘（内置广播）	RVV有`vfmul_vf`等效
`vmulq_lane_f32`	向量×lane广播乘	RVV无lane-indexed版本

指令	功能	RVV现状
`vmlaq_lane_f32`	FMA + lane广播	RVV无

lane-indexed语义:

```
; 从向量q2的低半部分(d2)中提取lane[1]广播到所有位置，与q0相乘
vmulq_lane_f32 q0, q1, d2[1]
```

收益分析:

场景	ARM NEON	RVV	差距
单次标量广播乘	1条 `vmulq_n_f32`	1条 `vfmul_vf`	相同
重复使用已加载alpha	1条 `vmulq_lane`	1条 `vfmul_vf` （重新广播）	RVV多1次广播开销

关键差异： - ARM的lane-indexed允许从向量寄存器中提取特定元素广播 - 可用于alpha已预加载到向量寄存器的情况 - RVV每次需从标量寄存器读取alpha

建议扩展: - `vfmul_lane vd, vs1, vs2, imm` — lane-indexed标量广播乘 - 语义: `vd[i] = vs1[i] * vs2[imm]`，从vs2提取指定lane广播

---

3. LoongArch LSX

核心特点： - 128-bit向量，4×float32 - `__lsx_vfmul_s`: 向量×向量乘法 - `__lsx_vreplgr2vr_w`: 标量广播到向量

指令序列:

```
vldrepl.w  vr1, alpha, 0    ; 广播alpha到向量（单独指令）
vfmul.s    vr2, vr0, vr1    ; 向量×向量乘
```

操作	LSX指令数	RVV指令数
标量广播	1	0（内置）
向量乘	1	1
总计	2	1

关键差异：RVV在此场景优于LSX（无需单独广播指令）。

---

4. Power VSX

核心特点： - 128-bit VSX寄存器 - `vec_mul`：向量×向量或向量×标量 - 广播通过`vec_splats`实现

与LSX类似，需单独广播指令。RVV在此场景有优势。

---

5. S390X

核心特点： - 128-bit Vector Facility - `vfma`：向量FMA - 广播需单独操作

与LSX/VSX类似，RVV在此场景有优势。

---

6. WASM SIMD

核心特点： - 128-bit v128 - `wasm_f32x4.mul`：向量×向量 - `wasm_f32x4.splat`：标量广播

与LSX类似，需单独广播指令。RVV有优势。

RVV扩展指令建议详细说明

[P0] vfmul_lane / vfmacc_lane

指令定义:

```
vfmul_lane vd, vs1, vs2, imm   # vd[i] = vs1[i] * vs2[imm]
vfmacc_lane vd, vs1, vs2, imm  # vd[i] += vs1[i] * vs2[imm]
```

应用场景： - alpha已预加载到向量寄存器 - 多次重复使用同一标量 - 避免每次从标量寄存器读取

性能对比:

方案	广播开销
当前RVV `vfmul_vf`	每次从标量寄存器读取
lane-indexed	一次加载，后续零广播开销

参考：ARM NEON `vmulq_lane_f32` / `vmlaq_lane_f32`

结论

关键发现

1. **RVV在标量广播乘场景优于大多数平台** - 相比x86 AVX：少1条预设置指令 - 相比LoongArch LSX：少1条广播指令 - `vfmul_vf`内置广播更简洁

2. **ARM NEON的lane-indexed乘法是唯一潜在优势** - 当alpha已预加载到向量寄存器时，可避免广播开销 - RVV每次需从标量寄存器读取

3. **MLAS原始实现使用标量循环** - ONNX Runtime CPU版QuickGelu alpha缩放是标量循环（未向量化） - RVV实现已显著优于原始scalar版本

优先级总结

优先级	扩展	收益范围	来源平台
P0	vfmul_lane	重复使用场景减少广播开销	ARM NEON

整体收益估算： - QuickGelu alpha缩放占整体执行约10%（基于profiling） - alpha缩放本身已简单（3指令/16元素） - lane-indexed扩展收益有限（仅当alpha预加载时）

审查日志

轮次	发现问题数	已修复	剩余
R1	6	6	0
R2	3	3	0

R1修复内容： 1. LSX元素计数修正：8×float32 → 4×float32 2. `__lsx_vfmul_s` 描述修正：向量×标量 → 向量×向量 3. LSX汇编助记符修正：xv→v（LASX→LSX） 4. 收益描述修正：50%指令减少 → 微架构层面广播开销减少 5. vfma_lane收益修正：同上 6. ARM NEON汇编语法修正：lane源使用D寄存器而非Q寄存器

R2修复内容： 1. BBV数据声明更新：从”未提供”改为引用QEMU-BBV profiling数据（output/bbv_rvv512/quick-gelu/） 2. 收益计算方式更新：从”无BBV数据”改为基于BBV热点数据估算，含整体收益公式 3. 新增BBV热点数据章节：核心循环BB指令序列、执行次数、标量路径占比及整体收益估算方法

最终审查结论：所有问题已修复。RVV vfmul_vf语义正确；ARM NEON lane-indexed指令语义正确；LSX/LASX区分明确；收益描述准确。